

Arquitectura y Diseño del Sistema: Asistente RAG de Nicho

Especificaciones Técnicas, Módulos y APIs

Agosto 2026

1. Introducción

Este documento define la arquitectura técnica y los requisitos para el desarrollo de una herramienta de Inteligencia Artificial especializada mediante **Generación Aumentada por Recuperación (RAG)**. El sistema está diseñado para ingerir documentos complejos (contratos legales, manuales técnicos) y permitir a los usuarios consultar información con alta precisión, reduciendo las alucinaciones del modelo base al obligarlo a fundamentar sus respuestas en los datos ingeridos.

2. Requisitos del Sistema

2.1 Requisitos Funcionales

- **Gestión de Documentos:** El sistema debe permitir la carga (upload), procesamiento y eliminación de documentos en formatos estándar (PDF, DOCX, TXT).
- **Búsqueda Semántica:** Capacidad de procesar consultas en lenguaje natural y recuperar los fragmentos de texto más relevantes.
- **Generación de Respuestas:** Síntesis de respuestas fluidas utilizando un Modelo de Lenguaje (LLM) contextualizado con los fragmentos recuperados.
- **Trazabilidad (Citas):** Cada respuesta generada debe incluir referencias exactas al documento de origen y la página.

2.2 Requisitos No Funcionales

- **Privacidad y Seguridad:** Los datos ingeridos no deben ser utilizados para entrenar modelos públicos. Se priorizarán LLMs alojados en servidores controlados o APIs con acuerdos estrictos de privacidad (Zero Data Retention).
- **Escalabilidad:** La base de datos vectorial debe soportar millones de *embeddings* con tiempos de búsqueda (latencia) inferiores a 500ms.
- **Modularidad:** El backend debe separar claramente los servicios de ingesta, almacenamiento y consulta.

3. Arquitectura y Módulos

La solución se estructura en cuatro módulos principales interactuando bajo un patrón de microservicios (o monolito modular) construido en **Python (FastAPI)**.

3.1 Módulo de Ingesta (Pipeline de Datos)

Responsable de transformar documentos crudos en vectores almacenables.

- **Extracción de Texto:** Uso de PyMuPDF o Unstructured para extraer texto, tablas y metadatos de PDFs.
- **Chunking (Segmentación):** División del texto en bloques manejables (ej. 500-1000 tokens) con un "solapamiento" (overlap) del 10% al 20% utilizando RecursiveCharacterTextSplitter de LangChain para no perder el contexto entre párrafos.
- **Vectorización (Embedding):** Conversión de cada "chunk" en un vector numérico de alta dimensionalidad (ej. 1536 dimensiones si se usa OpenAI text-embedding-3-small).

3.2 Módulo de Almacenamiento (Base de Datos Vectorial)

Gestión del índice semántico. Se utilizará **ChromaDB** (para despliegues locales/monolíticos) o **Pinecone** / **pgvector** (para entornos distribuidos).

- Almacena: ID del Documento, ID del Chunk, Vector, Texto Original, Metadatos (página, autor, fecha).

3.3 Módulo de Recuperación (Retrieval)

El motor de búsqueda semántica en tiempo real.

- Transforma la pregunta del usuario en un vector utilizando el mismo modelo de Embedding del Módulo 3.1.
- Ejecuta una búsqueda de Similitud del Coseno (Cosine Similarity) en la BD Vectorial para obtener el Top-K (ej. los 5 chunks más relevantes).
- *Optimización Avanzada:* Reranking (re-clasificación) de los resultados usando un modelo ligero de Cross-Encoder para mayor precisión.

3.4 Módulo de Generación (LLM)

El cerebro conversacional. Orquesta el ensamblaje del Prompt final.

- **Prompt Template:** *"Eres un asistente experto. Utiliza el siguiente CONTEXTO para responder la PREGUNTA. Si el contexto no contiene la respuesta, di 'No tengo información suficiente'."*
- Inyección del Top-K de chunks recuperados en el CONTEXTO.
- Llamada al LLM (ej. Llama 3 local vía Ollama o Claude 3.5 vía API).

4. Especificación de Endpoints (API RESTful)

El backend, construido con FastAPI, expondrá los siguientes endpoints para ser consumidos por el Frontend.

POST /api/v1/documents/ingest

Sube un archivo, extrae el texto, lo divide en chunks, genera los embeddings y los guarda en la base de datos vectorial.

Payload (multipart/form-data):

- file: Archivo PDF, DOCX, etc.
- metadata: JSON con etiquetas (departamento, categoría).

GET /api/v1/documents

Lista todos los documentos indexados en el sistema.

POST /api/v1/chat/query

Endpoint principal de consulta. Recibe la pregunta del usuario y devuelve la respuesta del LLM junto con las fuentes utilizadas.

Request JSON:

```
{ "query": "¿Cuáles son las condiciones de rescisión anticipada?",  
  "chat_history_id": "uuid-1234", "filters": { "category": "contratos_2026" } }
```

Response JSON:

```
{ "answer": "La rescisión anticipada requiere un preaviso de 30 días y...",  
  "sources": [ { "doc_id": "contrato_finca_x.pdf", "page": 12, "text_snippet":  
    "...el arrendatario deberá notificar con 30 días de antelación..." } ] }
```

5. Interfaces de Usuario (UI)

5.1 Interfaz de Usuario Final (Chat)

Una aplicación web (ej. construida con React/Next.js o Streamlit para prototipos rápidos) que imita la experiencia de ChatGPT, pero con diferencias clave:

- **Panel Principal:** Área de chat.
- **Panel Lateral de Fuentes:** Cuando la IA responde, los enlaces a los documentos citados aparecen a un lado. Al hacer clic, el PDF original se abre resaltando el párrafo exacto.

5.2 Interfaz de Administración (Backoffice)

Un panel exclusivo para administradores y técnicos de datos.

- **Gestor de Conocimiento:** Tabla CRUD para subir, borrar y actualizar documentos.
- **Dashboard de Analíticas:** Gráficos que muestran las preguntas más frecuentes que la IA *no* pudo responder (identificando brechas en la documentación).
- **Ajuste de Parámetros:** Controles (sliders) para ajustar el tamaño de los chunks o el *Top-K* de recuperación sin tocar el código.

6. Stack Tecnológico Recomendado (Resumen)

Componente	Tecnología Recomendada	Alternativa
Backend & API	Python / FastAPI	Node.js / Express
Orquestador RAG	LangChain	LlamaIndex
Base de Datos Vectorial	ChromaDB (Local)	Pinecone / Weaviate (Cloud)
Procesamiento de PDF	PyMuPDF	Unstructured.io
Modelo LLM (Razonamiento)	OpenAI GPT-4o / Claude 3.5	Llama 3 (Meta) vía Ollama
Frontend	React + TailwindCSS	Streamlit (Prototipado)